

Vorlese-Skript zur Präsentation

CloudOps Capstone · Danial Ahmed · Sprechzeit gesamt: ca. 6–8 Minuten · Die Texte stehen auch als Notizen in der PowerPoint-Datei

Folie 1 – Titel

Hallo zusammen, ich bin Danial Ahmed. Ich stelle euch heute mein Abschlussprojekt vor: das CloudOps Capstone. Dahinter steckt eine kleine Web-App – aber der eigentliche Star ist die komplett automatisierte DevOps-Pipeline: Vom git push bis zum fertigen Deployment auf einem AWS-Server läuft alles ohne Handarbeit.

Tipp: Langsam sprechen, Blickkontakt, kurz lächeln. Ca. 20 Sekunden.

Folie 2 – Die Idee

Die App selbst ist eine einfache Aufgaben-Verwaltung – Tickets anlegen, Status ändern, fertig. Ich habe sie bewusst klein gehalten. Denn mein Ziel war nicht die hundertste To-do-App, sondern das, was im Berufsalltag zählt: dass Software automatisch getestet, gebaut und ausgerollt wird. Genau das macht ein DevOps-Engineer jeden Tag.

Tipp: Beim Satz „bewusst klein gehalten“ ruhig eine Pause machen – das ist deine wichtigste Botschaft.

Folie 3 – Pipeline-Flow

So sieht der Weg des Codes aus. Erstens: Ich pushe eine Änderung auf den main-Branch. Zweitens: GitHub Actions startet automatisch und führt meine zehn pytest-Tests aus. Drittens: Nur wenn alle Tests grün sind, werden die Docker-Images gebaut – das nennt man Test-Gate. Und viertens wird die Staging-Umgebung auf AWS aktualisiert. Über jeden Lauf informiert mich Slack – Start, Erfolg oder Fehler. Für ein Produktions-Release setze ich einfach ein Versions-Tag wie v1.0.0 – das löst die Production-Pipeline aus.

Tipp: Mit dem Finger/Laserpointer die vier Kästen entlanggehen.

Folie 4 – Tech-Stack

Kurz zum Werkzeugkasten: Das Frontend ist React mit Vite, das Backend FastAPI in Python mit einer SQLite-Datenbank, die später gegen

PostgreSQL auf AWS getauscht werden kann. Beides läuft in Docker-Containern. Die komplette Infrastruktur – Netzwerk, Server, Speicher – ist mit Terraform als Code beschrieben. Die Pipeline läuft über GitHub Actions, und alle Zugangsdaten liegen verschlüsselt in GitHub Secrets – niemals im Code.

Tipp: Nicht jede Kachel vorlesen – die Folie zeigt sie ja. Zügig durch.

Folie 5 – Test-Gate

Qualität kommt zuerst. Zehn automatische Tests prüfen jeden API-Endpunkt – auch die Fehlerfälle, zum Beispiel wenn ein Titel fehlt oder eine ID nicht existiert. In der Pipeline dauert der komplette Testlauf nur zwölf Sekunden. Und die entscheidende Zeile in meiner Pipeline heißt: `needs test`. Das bedeutet: Der Deploy-Job darf erst starten, wenn der Test-Job erfolgreich war. Kaputter Code kommt so nie auf den Server.

Tipp: „`needs test`“ ist dein Fachbegriff-Moment – deutlich aussprechen.

Folie 6 – Docker

Die App wird in Docker-Container verpackt – nach dem Multi-Stage-Prinzip. Stufe eins baut die React-App mit Node. In Stufe zwei landen nur die fertigen Dateien plus der Nginx-Webserver. Das heißt: Im Produktions-Image ist kein Node und kein Quellcode – es ist klein und sicher. Das Backend läuft zusätzlich ohne Root-Rechte. Und mit einem einzigen Befehl, `docker compose up`, startet der komplette Stack – bei mir lokal genauso wie später auf dem AWS-Server.

Tipp: Falls Rückfrage „Warum Multi-Stage?“: kleineres Image, weniger Angriffsfläche, schnellere Deployments.

Folie 7 – Terraform

Die gesamte AWS-Infrastruktur existiert als Code – vier Terraform-Dateien. Sie beschreiben ein eigenes Netzwerk, die VPC, darin ein öffentliches Subnetz mit Internetzugang, und darin die EC2-Instanz – eine `t3.micro` im kostenlosen Free Tier. Die Firewall, also die Security Group, öffnet nur Port 80 für die Web-App und SSH für Deployments. Der Server installiert sich

beim ersten Start sogar selbst Docker. Wichtig: Terraforms Gedächtnis, der State, liegt zentral in einem S3-Bucket. Und wenn ich nicht übe, räume ich mit terraform destroy alles ab – so bleibt das Projekt praktisch kostenlos.

Tipp: Beim Kosten-Punkt kurz zur Kachel unten rechts zeigen – Kostenbewusstsein kommt bei Prüfern sehr gut an.

Folie 8 – Hürden & Learnings

Ganz ehrlich: Nicht alles lief glatt – und genau daraus habe ich am meisten gelernt. Mein Projekt liegt auf einer externen Festplatte, deren Ordnername einen Doppelpunkt enthält – das hat Python, npm und Vite gebrochen, weil der Doppelpunkt unter Unix Pfade trennt. Dann wurde die SQLite-Datenbank scheinbar zufällig schreibgeschützt – die Ursache: Auf exFAT-Platten verfallen offene Datei-Verbindungen nach etwa zehn Sekunden. Und der Docker-Build stolperte über versteckte macOS-Dateien. Jedes Problem habe ich gleich behandelt: systematisch eingrenzen, einen Minimal-Test bauen, die Lösung sauber dokumentieren. Genau diese Detektivarbeit ist DevOps-Alltag.

Tipp: Das ist deine stärkste Folie – hier zeigst du echtes Problemlösen. Zeit lassen!

Folie 9 – Status & Ausblick

Wo steht das Projekt heute? Fertig sind das Backend mit zehn grünen Tests, das React-Frontend, der komplette Docker-Stack und das GitHub-Repository mit laufender Test-Pipeline – der Test-Job läuft bereits bei jedem Push. Als Nächstes schalte ich mit Terraform die Staging-EC2 live, baue das echte Docker-Deployment in den Deploy-Job ein und aktiviere die Production-Pipeline über Versions-Tags. Der gesamte Fortschritt ist öffentlich auf GitHub nachvollziehbar – jeder Commit, jeder Pipeline-Lauf.

Tipp: Ehrlich bleiben, was noch offen ist – das wirkt souverän.

Folie 10 – Abschluss

Vielen Dank für eure Aufmerksamkeit! Wenn ihr mögt, zeige ich die Pipeline gerne live – ein git push genügt, und ihr seht die Tests und das Deployment in Echtzeit. Ich freue mich auf eure Fragen.

Tipp: Das Live-Demo-Angebot nur machen, wenn WLAN + Setup stehen. Vorher einmal proben!